VLSI Design Internship

Task 4: RTL Design of Finite State Machines (FSM) and Control Units

Submission contents
Moore FSM, Mealy FSM, traffic-light controller, overlapping 1011 sequence detector, self-checking testbenches, and online verification evidence.

Simulator used
EDA Playground - Icarus Verilog 12.0

Verification status
All four simulations passed. Each testbench also generated a VCD waveform file.

Task objective

Design synthesizable RTL finite-state machines in Verilog, verify reset and transition behavior with self-checking testbenches, and inspect the generated waveforms.

Core FSM concepts

Concept	Meaning used in this task
State	A stored condition of the controller, updated on a rising clock edge.
Moore FSM	Output depends only on the present state.
Mealy FSM	Output depends on the present state and the sampled input.
Reset	Asynchronous active-high reset that returns each design to a safe initial condition.

Online verification summary

Design	EDA Playground result	VCD created
3-state Moore FSM	PASS - S0 -> S1 -> S2 -> S0	moore_fsm.vcd
Input-based Mealy FSM	PASS - output behavior verified	mealy_fsm.vcd
Traffic-light controller	PASS - RED -> YELLOW -> GREEN -> RED	traffic_light.vcd
Overlapping 1011 detector	PASS - pattern detection verified	sequence_detector.vcd

Note: The state-to-light order intentionally follows the task specification: S0 = Red, S1 = Yellow, and S2 = Green.

Part B - Moore FSM (3-state cycle)

A three-state Moore machine cycles S0 -> S1 -> S2 -> S0. Its output is the registered state, so it changes only with the clock or reset.

Present state	Next state after rising clock edge
S0 (00)	S1 (01)
S1 (01)	S2 (10)
S2 (10)	S0 (00)
Invalid encoding	S0 (recovery)

Online simulator result

PASS: Moore FSM transitions S0 -> S1 -> S2 -> S0.

Synthesizable Verilog RTL

```
`timescale 1ns/1ps

// Part B: Moore FSM. Its visible output (state) depends only on the state.
module moore_fsm (
    input  wire       clk,
    input  wire       reset,
    output reg  [1:0] state
);
    localparam S0 = 2'b00;
    localparam S1 = 2'b01;
    localparam S2 = 2'b10;

    always @(posedge clk or posedge reset) begin
        if (reset)
            state <= S0;
        else begin
            case (state)
                S0:     state <= S1;
                S1:     state <= S2;
                S2:     state <= S0;
                default: state <= S0;
            endcase
        end
    end
endmodule
```

Self-checking testbench

```
`timescale 1ns/1ps
module tb_moore_fsm;
    reg clk = 1'b0;
    reg reset = 1'b0;
    wire [1:0] state;
    integer errors = 0;

    moore_fsm dut (.clk(clk), .reset(reset), .state(state));
    always #5 clk = ~clk;

    task expect_state;
        input [1:0] expected;
        begin
            #1;
            if (state !== expected) begin
                $display("FAIL: expected state %b, got %b at t=%0t", expected, state, $time);
                errors = errors + 1;
            end
        end
    endtask

    initial begin
        $dumpfile("moore_fsm.vcd");
        $dumpvars(0, tb_moore_fsm);
        #2 reset = 1'b1; #1 expect_state(2'b00);
        #7 reset = 1'b0;
        @(posedge clk); expect_state(2'b01);
        @(posedge clk); expect_state(2'b10);
        @(posedge clk); expect_state(2'b00);
        if (errors == 0) $display("PASS: Moore FSM transitions S0 -> S1 -> S2 -> S0.");
        else $display("FAIL: Moore FSM had %0d error(s).", errors);
        $finish;
    end
endmodule
```

Part C - Mealy FSM (input-based output)

This two-state Mealy FSM asserts the registered output only when the FSM is already in S1 and the sampled input is 1.

Present state	Input	Next state	Output after clock
S0	0	S0	0
S0	1	S1	0
S1	0	S0	0
S1	1	S1	1

Online simulator result

PASS: Mealy FSM output behavior verified.

Synthesizable Verilog RTL

```
`timescale 1ns/1ps

// Part C: registered Mealy FSM. At a clock edge, out is set from the
// pre-edge state and input. It is high only for state S1 with in = 1.
module mealy_fsm (
    input  wire clk,
    input  wire reset,
    input  wire in,
    output reg  out
);
    reg state;
    localparam S0 = 1'b0;
    localparam S1 = 1'b1;

    always @(posedge clk or posedge reset) begin
        if (reset) begin
            state <= S0;
            out   <= 1'b0;
        end else begin
            case (state)
                S0: begin
                    state <= in ? S1 : S0;
                    out   <= 1'b0;
                end
                S1: begin
                    state <= in ? S1 : S0;
                    out   <= in;
                end
                default: begin
                    state <= S0;
                    out   <= 1'b0;
                end
            endcase
        end
    end
endmodule
```

Self-checking testbench

```
`timescale 1ns/1ps
module tb_mealy_fsm;
    reg clk = 1'b0;
    reg reset = 1'b0;
    reg in = 1'b0;
    wire out;
    integer errors = 0;

    mealy_fsm dut (.clk(clk), .reset(reset), .in(in), .out(out));
    always #5 clk = ~clk;

    task expect_out;
        input expected;
        begin
            #1;
            if (out !== expected) begin
                $display("FAIL: expected out %b, got %b at t=%0t", expected, out, $time);
                errors = errors + 1;
            end
        end
    endtask

    initial begin
        $dumpfile("mealy_fsm.vcd");
        $dumpvars(0, tb_mealy_fsm);
        #2 reset = 1'b1; #1 expect_out(1'b0);
        #7 reset = 1'b0;
        @(negedge clk) in = 1'b1; @(posedge clk); expect_out(1'b0); // enter S1
        @(negedge clk) in = 1'b1; @(posedge clk); expect_out(1'b1); // S1 and in=1
        @(negedge clk) in = 1'b0; @(posedge clk); expect_out(1'b0); // leave S1
        if (errors == 0) $display("PASS: Mealy FSM output behavior verified.");
        else $display("FAIL: Mealy FSM had %0d error(s).", errors);
        $finish;
    end
endmodule
```

Part D - Traffic-light controller

A Moore controller maps each of the three registered states to one one-hot light output. Reset produces Red immediately.

State	Light output	Next state
S0	RED (100)	S1
S1	YELLOW (010)	S2
S2	GREEN (001)	S0

Online simulator result

PASS: Traffic light RED -> YELLOW -> GREEN -> RED.

Synthesizable Verilog RTL

```
`timescale 1ns/1ps

// Part D: Moore traffic-light controller.
// The required mapping is S0 = red, S1 = yellow, S2 = green.
module traffic_light (
    input  wire       clk,
    input  wire       reset,
    output reg  [2:0] light
);
    localparam RED    = 3'b100;
    localparam YELLOW = 3'b010;
    localparam GREEN  = 3'b001;

    localparam S0 = 2'b00;
    localparam S1 = 2'b01;
    localparam S2 = 2'b10;

    reg [1:0] state;

    always @(posedge clk or posedge reset) begin
        if (reset)
            state <= S0;
        else begin
            case (state)
                S0:     state <= S1;
                S1:     state <= S2;
                S2:     state <= S0;
                default: state <= S0;
            endcase
        end
    end

    always @(*) begin
        case (state)
            S0:     light = RED;
            S1:     light = YELLOW;
            S2:     light = GREEN;
            default: light = RED;
        endcase
    end
endmodule
```

Self-checking testbench

```
`timescale 1ns/1ps
module tb_traffic_light;
    reg clk = 1'b0;
    reg reset = 1'b0;
    wire [2:0] light;
    integer errors = 0;

    localparam RED    = 3'b100;
    localparam YELLOW = 3'b010;
    localparam GREEN  = 3'b001;
    traffic_light dut (.clk(clk), .reset(reset), .light(light));
    always #5 clk = ~clk;

    task expect_light;
        input [2:0] expected;
        begin
            #1;
            if (light !== expected) begin
                $display("FAIL: expected light %b, got %b at t=%0t", expected, light, $time);
                errors = errors + 1;
            end
        end
    endtask

    initial begin
        $dumpfile("traffic_light.vcd");
        $dumpvars(0, tb_traffic_light);
        #2 reset = 1'b1; #1 expect_light(RED);
        #7 reset = 1'b0;
        @(posedge clk); expect_light(YELLOW);
        @(posedge clk); expect_light(GREEN);
        @(posedge clk); expect_light(RED);
        if (errors == 0) $display("PASS: Traffic-light sequence RED -> YELLOW -> GREEN -> RED.");
        else $display("FAIL: Traffic-light controller had %0d error(s).", errors);
        $finish;
    end
endmodule
```

Part E - Overlapping 1011 sequence detector

A four-state Mealy detector tracks useful prefixes of 1011. When the final 1 arrives, detected pulses for one clock period and the machine returns to S1 so an overlap can begin with that same final 1.

State meaning	Input 0	Input 1
S0: no prefix	S0	S1
S1: saw 1	S2	S1
S2: saw 10	S0	S3
S3: saw 101	S2	S1 + detected pulse

Online simulator result

PASS: Overlapping 1011 detector verified.

Synthesizable Verilog RTL

```
`timescale 1ns/1ps

// Part E: overlapping Mealy detector for the serial pattern 1011.
// detected pulses for one clock period on the final '1' of every match.
module sequence_detector (
    input  wire clk,
    input  wire reset,
    input  wire in,
    output reg  detected
);
    localparam S0 = 2'b00;  // no useful prefix
    localparam S1 = 2'b01;  // saw 1
    localparam S2 = 2'b10;  // saw 10
    localparam S3 = 2'b11;  // saw 101

    reg [1:0] state;

    always @(posedge clk or posedge reset) begin
        if (reset) begin
            state    <= S0;
            detected <= 1'b0;
        end else begin
            detected <= 1'b0;  // default: a one-cycle pulse, not a sticky flag
            case (state)
                S0:     state <= in ? S1 : S0;
                S1:     state <= in ? S1 : S2;
                S2:     state <= in ? S3 : S0;
                S3: begin
                    if (in) begin
                        detected <= 1'b1;
                        state    <= S1;  // retain the final 1 for overlap
                    end else
                        state <= S2;     // 1010 retains the prefix 10
                end
                default: state <= S0;
            endcase
        end
    end
endmodule
```

Self-checking testbench

```
`timescale 1ns/1ps
module tb_sequence_detector;
    reg clk = 1'b0;
    reg reset = 1'b0;
    reg in = 1'b0;
    wire detected;
    integer errors = 0;

    sequence_detector dut (.clk(clk), .reset(reset), .in(in), .detected(detected));
    always #5 clk = ~clk;

    task send_bit;
        input value;
        input expected_detected;
        begin
            @(negedge clk) in = value;
            @(posedge clk); #1;
            if (detected !== expected_detected) begin
                $display("FAIL: input %b expected detected %b, got %b at t=%0t", value,
expected_detected, detected, $time);
                errors = errors + 1;
            end
        end
    endtask

    initial begin
        $dumpfile("sequence_detector.vcd");
        $dumpvars(0, tb_sequence_detector);
        #2 reset = 1'b1; #1;
        if (detected !== 1'b0) begin errors = errors + 1; $display("FAIL: reset did not clear
detected"); end
        #7 reset = 1'b0;

        // 1011 has one match on the fourth sampled bit.
        send_bit(1'b1, 1'b0); send_bit(1'b0, 1'b0);
        send_bit(1'b1, 1'b0); send_bit(1'b1, 1'b1);

        // 1011011 overlaps: the prior final 1 begins the next candidate.
        send_bit(1'b0, 1'b0); send_bit(1'b1, 1'b0);
        send_bit(1'b1, 1'b1);

        if (errors == 0) $display("PASS: overlapping 1011 detector verified.");
        else $display("FAIL: sequence detector had %0d error(s).", errors);
        $finish;
    end
endmodule
```

Part F and G - Testbench and waveform analysis

Every supplied testbench creates a clock, applies reset, drives the required stimulus, checks the expected output, and writes a VCD dump. The online runs used EDA Playground with Icarus Verilog 12.0.

Sequence-detector waveform expectation for input stream 1011011

Sampled bit number	1	2	3	4	5	6	7
Input in	1	0	1	1	0	1	1
detected	0	0	0	1	0	0	1

The pulses at samples 4 and 7 demonstrate overlapping detection: the final 1 of the first 1011 is retained as the first 1 of a possible next pattern.

Submission checklist

Item	Status
Moore FSM RTL and testbench	Completed and PASS
Mealy FSM RTL and testbench	Completed and PASS
Traffic-light RTL and testbench	Completed and PASS
Overlapping 1011 detector RTL and testbench	Completed and PASS
VCD waveform generated for each simulation	Completed
Online simulator used	EDA Playground - Icarus Verilog 12.0

End of submission.